AAMSIR Technical Report

Adaptive Architecture for Multi-Strategy
Information Retrieval

Team 5 | S26CS6.401 — Software Engineering | Project 3

Team Member	Roll Number
Arihant Tripathy	2023111026
Aviral Gupta	2023111023
Inesh Dheer	2023111010
Mohit Kumar Singh	2023111021
Sreemaee Akshathala	2025701028

GitHub Repository:

https://github.com/Arihant25/aamsir

April 2026

Contents

1 Introduction **2**
- 1.1 Problem Statement . . . 2
- 1.2 Solution: AAMSIR . . . 2
- 1.3 Technology Stack . . . 2

2 Task 1: Requirements and Subsystems **2**
- 2.1 Functional Requirements . . . 2
- 2.2 Non-Functional Requirements . . . 3
- 2.3 Architecturally Significant Requirements . . . 3
- 2.4 Subsystem Overview . . . 3

3 Task 2: Architecture Framework **5**
- 3.1 Stakeholder Identification (IEEE 42010) . . . 5
  - 3.1.1 Viewpoints and Views . . . 5
- 3.2 Architecture Decision Records (ADRs) . . . 5
  - 3.2.1 ADR-001: Microkernel for Retrieval Engine . . . 5
  - 3.2.2 ADR-002: Pipe-and-Filter for Ingestion . . . 5
  - 3.2.3 ADR-003: Strategy Pattern for Retrieval Algorithms . . . 6
  - 3.2.4 ADR-004: Broker Pattern with Scheduler . . . 6

4 Task 3: Architectural Tactics and Patterns **6**
- 4.1 Architectural Tactics . . . 6
- 4.2 Implementation Patterns . . . 7
  - 4.2.1 Pattern 1: Strategy Pattern . . . 7
  - 4.2.2 Pattern 2: Chain of Responsibility . . . 7

5 Task 4: Prototype Implementation and Analysis **7**
- 5.1 Prototype Overview . . . 7
- 5.2 Key Implementation Details . . . 8
  - 5.2.1 Document Ingestion (Pipe-and-Filter + Adapter) . . . 8
  - 5.2.2 Retrieval Engine (Microkernel + Strategy) . . . 8
  - 5.2.3 Context Aggregation and Enrichment . . . 8
  - 5.2.4 Conversational RAG with Query Rewriting (FR-07) . . . 9
  - 5.2.5 Frontend . . . 9
- 5.3 Architecture Analysis . . . 9
  - 5.3.1 NFR-01: Latency (Measured) . . . 9
  - 5.3.2 NFR-05: Modifiability (Quantified) . . . 10
  - 5.3.3 Trade-offs . . . 10
- 5.4 Design Patterns Summary . . . 10

6 Reflections and Lessons Learned **11**
- 6.1 Architectural Decisions . . . 11
- 6.2 Technology Choices . . . 11
- 6.3 Challenges . . . 11
- 6.4 Future Improvements . . . 11

7 Individual Contributions **12**

8 References **12**

Introduction

Problem Statement

Institutions accumulate large volumes of internal documents — policies, SOPs, guidelines, and knowledge bases — that employees frequently need to query. Existing solutions are either too shallow (keyword search misses semantically related content) or privacy-compromising (cloud-hosted LLMs require sensitive documents to leave the organization's network).

Solution: AAMSIR

AAMSIR (Adaptive Architecture for Multi-Strategy Information Retrieval) addresses this gap with a chatbot-style interface backed by three interchangeable retrieval strategies — syntactic, semantic, and agentic — all running on compact, on-premise models to preserve institutional privacy.

Technology Stack

Layer	Technology
Backend	Python 3.11+, FastAPI, uv (package manager)
Vector Database	ChromaDB (embedded, on-premise)
Relational Database	SQLite (via SQLAlchemy)
Embeddings	sentence-transformers (all-MiniLM-L6-v2)
Keyword Search	rank_bm25
LLM (Optional)	Ollama (qwen2.5:7b)
Frontend	Next.js 16, TypeScript, Tailwind CSS
PDF Extraction	pdfplumber

Table 1: Technology Stack

Task 1: Requirements and Subsystems

Functional Requirements

ID	Requirement	Architectural Significance
FR-01	Query Interface Dashboard	Drives the Facade pattern (single API entry point)
FR-02	Result Exploration	Requires chunked storage, metadata tracking, and file-serving endpoint
FR-03	User Feedback Module	Requires persistence layer and analytics
FR-04	Multi-Modal Retrieval	Drives Strategy Pattern and Microkernel
FR-05	Retrieval Configuration Panel	Drives runtime strategy switching
FR-06	Document Management Module	Drives Pipe-and-Filter ingestion pipeline
FR-07	Conversational Query Refinement	Drives two-phase LLM invocation: rewrite → retrieve → generate

Table 2: Functional Requirements

Non-Functional Requirements

ID	Requirement	Target
NFR-01	Latency	P95 $\leq$ 15 seconds
NFR-02	Throughput	$\geq$ 50 concurrent users
NFR-03	Scalability	10,000 indexed documents
NFR-04	Availability	$\geq$ 99% during working hours
NFR-05	Modifiability	New strategies in $<$ 100 person-hours
NFR-06	Data Security	100% on-premise, AES-256 at rest

Table 3: Non-Functional Requirements

Architecturally Significant Requirements

- **ASR-01 (Extensibility):** Drove adoption of Microkernel + Strategy Pattern. The NLP field evolves rapidly; new retrieval methods must be addable without rewriting the core system.
- **ASR-02 (Latency via Context Limitation):** Drove Chain of Responsibility in semantic retrieval. SLMs have finite context windows; feeding all retrieved documents would violate latency constraints.
- **ASR-03 (Scalability of Ingestion):** Drove Pipe-and-Filter architecture. Processing 10,000 documents involves heavy compute steps that can bottleneck the system.
- **ASR-04 (Fault Tolerance):** Drove plugin sandboxing in the Microkernel. Agentic retrieval involves complex tool execution which may hang or fail.

Subsystem Overview

The system is divided into five subsystems:

1. **Presentation Layer:** Next.js web dashboard with query interface, document management, and settings.
2. **Document Ingestion Pipeline:** Pipe-and-Filter architecture for Extract $\rightarrow$ Summarize $\rightarrow$ Chunk $\rightarrow$ Embed $\rightarrow$ Index.
3. **Hybrid Retrieval Engine:** Microkernel with Syntactic, Semantic, and Agentic strategy plugins.
4. **Generation & Orchestration Layer:** Orchestrator (Facade), Context Aggregator (RRF), answer generation.
5. **Data Persistence Layer:** SQLite for metadata/feedback, ChromaDB for vector embeddings.

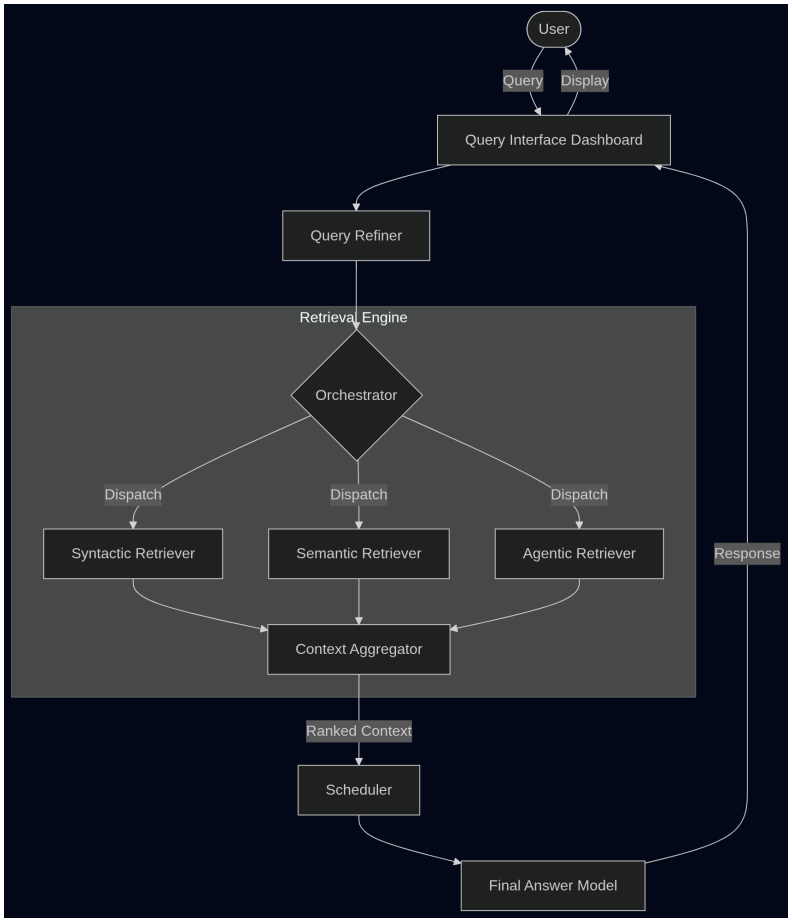

Figure 1: High-level architecture showing the flow of a user query through the Presentation, Retrieval Engine, and Generation layers.

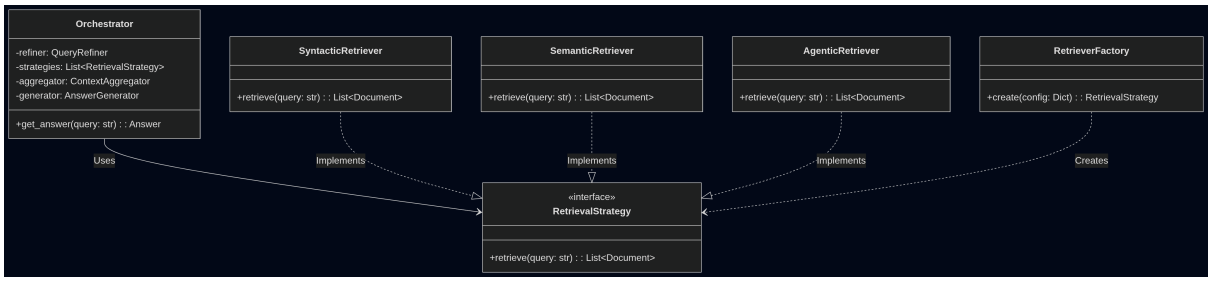

Figure 2: UML class diagram of the retrieval engine: Orchestrator (Facade), RetrievalStrategy (interface), concrete retrievers, and RetrieverFactory.

Task 2: Architecture Framework

Stakeholder Identification (IEEE 42010)

Stakeholder	Concerns
End Users (Researchers/Students)	Precision, Latency, Usability, Trustworthiness
System Administrators	Deployability, Availability, Resource Efficiency, Observability
Course Instructors/Evaluators	Compliance to Standards, Modularity, Design Pattern Correctness
Developers/Maintenance Team	Modifiability, Testability, Maintainability
Data Owners/Content Providers	Data Integrity, Confidentiality

Table 4: Stakeholders and Concerns (IEEE 42010)

Viewpoints and Views

Four viewpoints are used to address stakeholder concerns:

1. **Functional / Logical Viewpoint:** Describes what the system does — the Orchestrator facade, strategy plugins, and context aggregation logic. Addressed via UML class and sequence diagrams.
2. **Development / Module Viewpoint:** Describes the package decomposition (`api/`, `ingestion/`, `retrieval/`, `database.py`, `config.py`). Each plugin depends only on the `RetrievalStrategy` interface.
3. **Deployment / Operational Viewpoint:** All components run on-premise: Next.js frontend, FastAPI backend, SQLite, ChromaDB, and Ollama within the institution's network boundary.
4. **Information / Data Viewpoint:** Documents transition through three representations: raw upload (filesystem), structured record (SQLite), and vector index (ChromaDB).

Architecture Decision Records (ADRs)

All ADRs follow the Nygard template (Status, Context, Decision Drivers, Considered Options, Decision Outcome, Pros/Cons).

ADR-001: Microkernel for Retrieval Engine

- **Context:** Multiple evolving retrieval strategies require independent addition/removal.
- **Decision:** Adopt Microkernel (Plugin) Architecture.
- **Consequences:** Open/Closed Principle satisfied; fault isolation; simplified testing.
- **Alternatives Rejected:** Monolithic Controller (violates OCP), Microservices (excessive overhead).

ADR-002: Pipe-and-Filter for Ingestion

- **Context:** Document ingestion involves fixed-order transformations that need flexible step swapping.
- **Decision:** Implement Pipe-and-Filter Architecture.
- **Consequences:** Loose coupling, supports parallelism, maintainability.

- **Alternatives Rejected:** Batch Script (monolithic failure), Event-Driven (excessive complexity).

ADR-003: Strategy Pattern for Retrieval Algorithms

- **Context:** Three different retrieval algorithms need runtime interchangeability.
- **Decision:** Implement Strategy Pattern with `RetrievalStrategy` interface.
- **Consequences:** Clean separation, independently testable, OCP compliance.
- **Alternatives Rejected:** Conditional Logic (violates OCP), Template Method (algorithms differ too much).

ADR-004: Broker Pattern with Scheduler

- **Context:** Multiple concurrent requests competing for limited GPU resources.
- **Decision:** Implement Broker Pattern with task queue and worker pool.
- **Consequences:** Asynchronous processing, load management, resilience.
- **Alternatives Rejected:** Direct Sync Calls (blocking), Observer Pattern (inefficient load balancing).

Task 3: Architectural Tactics and Patterns

Architectural Tactics

#	Tactic	Category	NFR Addressed
1	Caching Proxy	Performance	NFR-01 (Latency $\leq$ 15s P95)
2	Cascading / Progressive Narrowing	Performance	NFR-01 (Latency $\leq$ 15s P95)
3	Fault Isolation via Plugin Sandboxing	Availability	NFR-04 (Uptime $\geq$ 99%)
4	Horizontal Scaling of Stateless Filters	Scalability	NFR-03 (10K documents)
5	Task Queuing with Resource Pooling	Throughput	NFR-02 (50 concurrent users)

Table 5: Architectural Tactics

Tactic 1 — Caching (Proxy): The `CachingAgenticProxy` wraps the expensive `AgenticRetriever` with an LRU cache (128 entries, MD5-keyed). Cache hits return in <1ms instead of the 5–30s agentic retrieval time.

Tactic 2 — Cascading / Progressive Narrowing: The `SemanticRetriever` implements a Chain of Responsibility pipeline with three stages: TitleMatch (very low cost), SummarySim (low cost), ContentSim (medium cost). This reduces full-content vector comparisons from $\sim$10,000 to $\sim$75, a 99.25% reduction.

Tactic 3 — Fault Isolation: The Orchestrator wraps each plugin invocation in an exception boundary. If a plugin crashes or times out, the kernel logs the failure and returns partial results from healthy plugins.

Tactic 4 — Horizontal Scaling: The ingestion pipeline's stateless filters can be replicated across multiple workers. The Embedder filter (most compute-intensive) can scale to N workers consuming from a shared queue.

Tactic 5 — Task Queuing with Resource Pooling: The Scheduler component acts as a Broker between the Orchestrator and model inference workers, buffering requests in a FIFO queue to smooth traffic spikes.

Implementation Patterns

Pattern 1: Strategy Pattern

The Strategy Pattern defines a common `RetrievalStrategy` interface for a family of retrieval algorithms. Concrete strategy classes (`SyntacticRetriever`, `SemanticRetriever`, `CachingAgenticProxy`) implement the interface. The `Orchestrator` holds a collection of strategy objects and delegates retrieval to them without any type checks or conditionals.

How it works:

1. At startup, concrete strategies are created via `StrategyFactory.create()` and registered with the Orchestrator.
2. When `query(query_text)` is called, the Orchestrator iterates over its strategies list and calls `retrieve(query)` on each.
3. Results are passed to the `ContextAggregator`, which deduplicates and re-ranks them using Reciprocal Rank Fusion.
4. Adding a new strategy requires only implementing `RetrievalStrategy` and adding a factory case. The Orchestrator code is *not modified*.

Consequences: New retrieval strategies can be added without modifying the Orchestrator (OCP). Each strategy is independently unit-testable. Strategies can be enabled/disabled at runtime via the configuration panel.

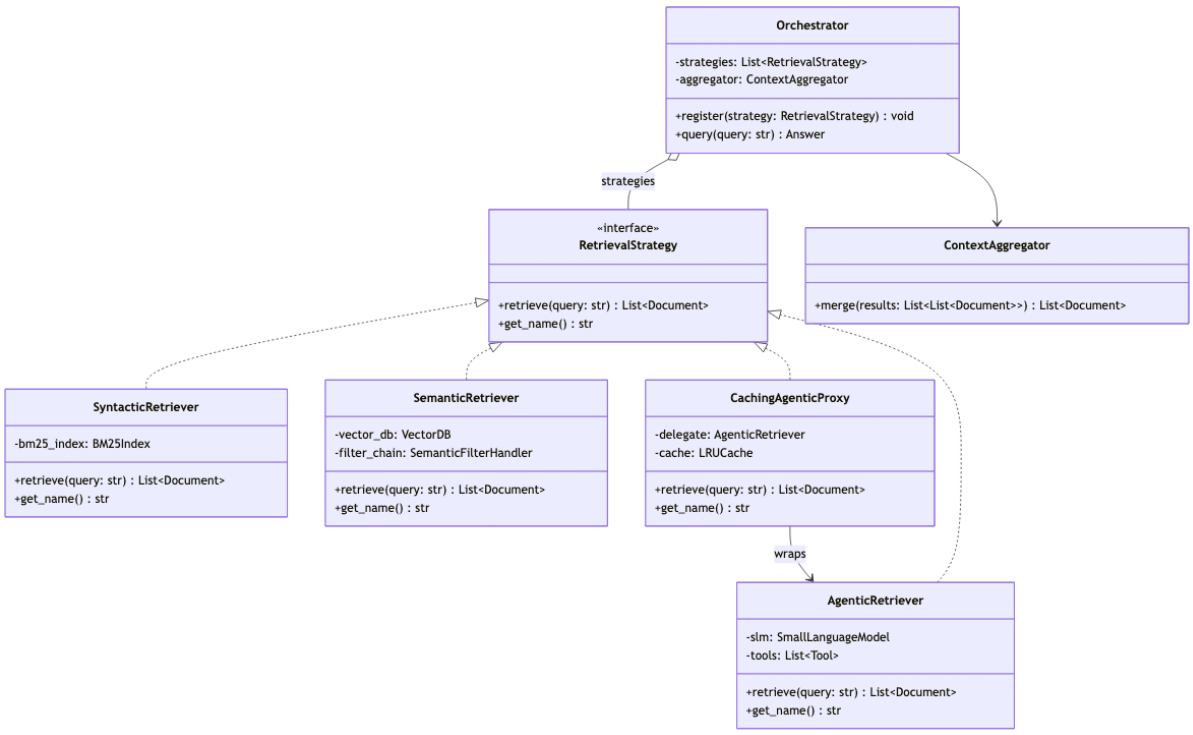

Figure 3: Strategy Pattern — Class Diagram

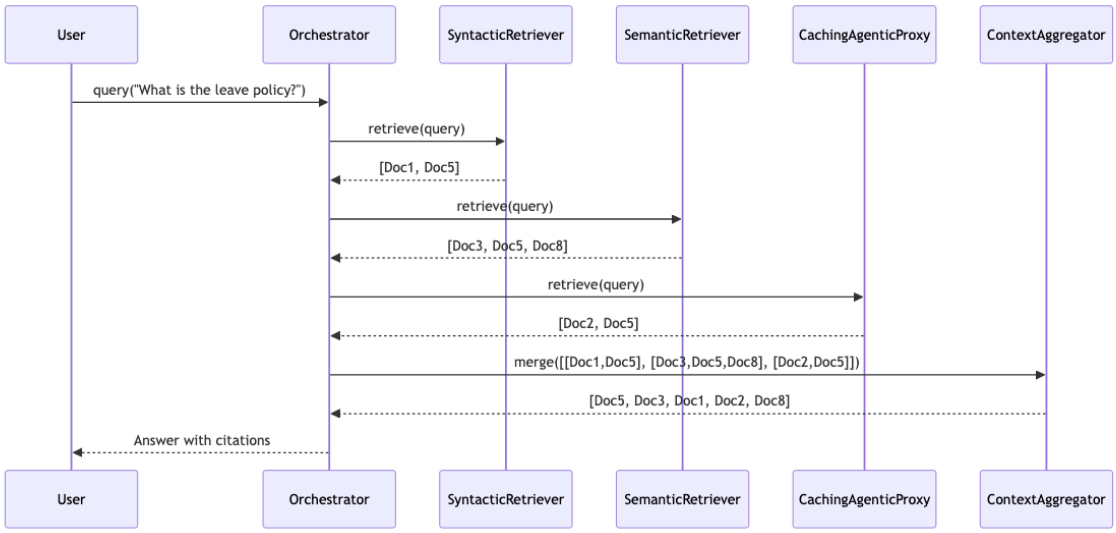

Figure 4: Strategy Pattern — Sequence Diagram (sequential invocation)

Pattern 2: Chain of Responsibility

The Chain of Responsibility implements a graduated filtering pipeline within the `SemanticRetriever`. An abstract `SemanticFilterHandler` defines the chain interface.

Chain structure:

1. `TitleMatchHandler` — Lexical overlap between query tokens and document titles. Input: ∼10,000 docs → Output: ∼800 docs. Cost: Very Low.
2. `SummarySimHandler` — Cosine similarity on pre-computed summary embeddings. Input: ∼800 docs → Output: ∼75 docs. Cost: Low.
3. `ContentSimHandler` — Full vector search via ChromaDB on filtered candidates. Input: ∼75 docs → Output: ∼15 docs. Cost: Medium.

Consequences: Dramatically reduces latency of semantic search (99%+ reduction in expensive comparisons). Each handler can be independently tuned. New filtering stages can be inserted anywhere in the chain.

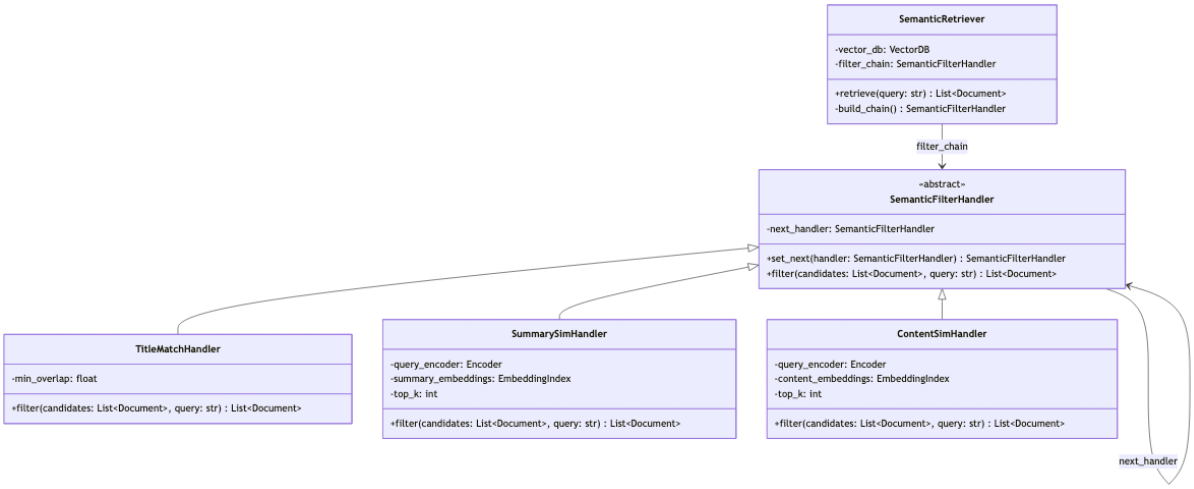

Figure 5: Chain of Responsibility — Class Diagram

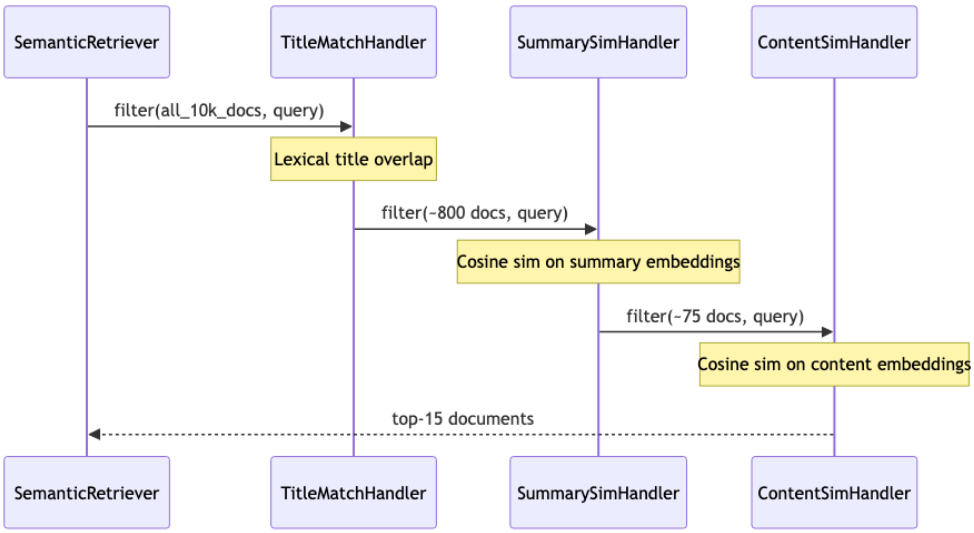

Figure 6: Chain of Responsibility — Sequence Diagram

Task 4: Prototype Implementation and Analysis

Prototype Overview

The prototype implements a **complete end-to-end query-answer pipeline**:

1. User uploads documents via the web dashboard.
2. Documents are ingested through the Pipe-and-Filter pipeline (Extract → Summarize → Chunk → Embed → Index).
3. User submits natural-language queries.
4. The Orchestrator invokes selected retrieval strategies sequentially with fault isolation.
5. Results are merged using Reciprocal Rank Fusion (RRF).
6. An answer is generated (LLM if Ollama available, extractive otherwise).

7. Sources with citations are displayed.
8. User can provide feedback (helpful/not helpful).

Key Implementation Details

Document Ingestion (Pipe-and-Filter + Adapter)

The ingestion pipeline implements ADR-002 as five stateless filter functions operating on a `DocumentEnvelope`:

```
extract_text() -> summarize() -> chunk_text() -> embed_chunks() -> index_document()
```

The text extraction filter uses the **Adapter Pattern** (`ExtractorRegistry`) to dynamically select the correct format handler at runtime. Adding support for a new format (e.g., `.docx`) requires only implementing the `TextExtractor` interface and registering it — no pipeline code changes.

Retrieval Engine (Microkernel + Strategy)

The `Orchestrator` manages three retrieval plugins registered via `StrategyFactory`:

- **SyntacticRetriever:** Rebuilds a BM25 index over paragraph-level chunks at query time. Applies a 50%-of-top-score threshold to filter noise. Average latency: ∼2ms.
- **SemanticRetriever:** Adaptive — for corpora under 100 documents it queries ChromaDB directly with a 60%-similarity cutoff; for larger corpora it engages the full Chain of Responsibility pipeline, reducing candidate sets by ∼99%. Average steady-state latency: ∼72ms.
- **CachingAgenticProxy:** Wraps `AgenticRetriever` (Ollama `qwen2.5:7b`) with an LRU cache (128 entries, MD5-keyed). Cache hits return in <1ms. The agentic retriever uses Ollama tool-calling to explore the document corpus across multiple inference rounds.

Each plugin is invoked inside an exception boundary (Tactic 3). If a plugin crashes or times out, the Orchestrator logs the failure and returns partial results from healthy plugins.

Context Aggregation and Enrichment

The `ContextAggregator` merges results from all active strategies using **Reciprocal Rank Fusion (RRF)** with constant $k = 60$. Each document's aggregated score is:

$$score(d) = \sum_{s \in strategies} \frac{1}{k + rank_s(d)}$$

After merging, a relevance filter retains only chunks scoring ≥70% of the top RRF score. For each relevant chunk, full document content (up to 2,000 chars) is fetched from SQLite for richer context.

Conversational RAG with Query Rewriting (FR-07)

The Orchestrator implements a two-phase LLM pipeline when conversation history is present:

1. **Rewrite:** The LLM rewrites the follow-up question into a standalone search query using the last 6 messages of history.
2. **Retrieve:** Retrieval runs using the rewritten query (BM25 + vector).

3. **Generate:** Answer generation uses the original query + history + retrieved context.

The rewrite step is skipped for first messages (no history), adding zero overhead. The streaming endpoint (`POST /api/query/stream`) emits four SSE event types: `rewrite`, `sources`, `token`, `done`. This phased streaming lets the frontend render cited sources immediately while the answer is still generating.

Frontend

The Next.js frontend provides three pages:

- **Query Dashboard:** Streaming chatbot interface with per-message strategy toggles, expandable source cards, and RewriteChip hover card.
- **Document Management:** Upload, listing with search, delete; document preview via inline file serving.
- **Settings & Analytics:** Strategy enable/disable toggles, model configuration, real-time usage statistics.

Architecture Analysis

We compared the implemented **Microkernel + Strategy** architecture against a hypothetical **Monolithic Controller** alternative.

NFR-01: Latency (Measured)

Hardware: MacBook Pro with Apple M1 Pro, 16 GB RAM, macOS. All inference (embedding model + optional SLM) runs locally on-device using MPS (Metal Performance Shaders).

Benchmark: 20 diverse queries against an 8-document corpus. Results from `benchmark.py`:

Configuration	Mean	P50	P95	Throughput	NFR-01
Syntactic only	13.5ms	13.3ms	14.4ms	73.8 q/s	PASS
Semantic only	491.5ms	17.3ms	787.3ms	2.0 q/s	PASS
Syntactic + Semantic	30.1ms	29.8ms	32.9ms	33.2 q/s	PASS

Table 6: Latency Benchmark Results (NFR-01: P95 $\leq$ 15s)

All configurations pass NFR-01 by a wide margin. **Note:** These benchmarks measure retrieval latency only. End-to-end response generation via the Ollama SLM adds approximately 5–8 seconds depending on the model chosen (e.g., qwen2.5:7b), as the LLM must synthesize an answer from the retrieved context. This generation time is independent of the retrieval architecture.

The monolithic architecture would achieve similar raw latency, but loses the fault isolation benefit: a crash in one retrieval path would fail the entire query.

NFR-05: Modifiability (Quantified)

We measure modifiability by counting files/lines changed to add a hypothetical new retrieval strategy (e.g., Graph RAG).

Metric	Microkernel + Strategy	Monolithic
Files to modify	1 new + 1 modified	2–3 modified
Lines of code changed	∼63	∼80–100
OCP compliance	Yes	No
Estimated effort	4–8 person-hours	16–24 person-hours
Isolated unit testing	Yes	No

Table 7: Modifiability Comparison (NFR-05)

Trade-offs

Attribute	Microkernel + Strategy	Monolithic
Raw latency	Equivalent (∼74ms dual-strategy steady-state)	Equivalent
Fault tolerance	Partial results on plugin crash	Total query failure
Modifiability	4–8 hrs, OCP compliant	16–24 hrs, modifies core code
Testability	Isolated per-strategy	Integration testing required
Initial complexity	Higher (interfaces, registry, factory)	Lower (simpler initial implementation)
Dispatch overhead	∼2ms	None

Table 8: Architecture Trade-off Analysis

Conclusion: The Microkernel + Strategy architecture is the correct choice because extensibility (ASR-01) is the primary quality driver, and fault isolation (ASR-04) prevents the unreliable agentic retriever from degrading the overall system. The ∼2ms dispatch overhead is negligible compared to the 72–348ms retrieval and embedding operations measured in our benchmarks.

Design Patterns Summary

Pattern	File	Purpose
Strategy	`retrieval/strategy.py`	`RetrievalStrategy` ABC; polymorphic retrieval
Microkernel	`retrieval/orchestrator.py`	Plugin registry, fault-isolated dispatch, RRF
Factory	`retrieval/factory.py`	`StrategyFactory.create()` decouples instantiation
Chain of Resp.	`retrieval/semantic.py`	TitleMatch → SummarySim → ContentSim cascade
Adapter	`ingestion/extractors.py`	`ExtractorRegistry` maps extensions to extractors
Proxy (Caching)	`retrieval/agentic.py`	LRU cache wrapping `AgenticRetriever`
Pipe-and-Filter	`ingestion/pipeline.py`	Five stateless filters on `DocumentEnvelope`
Facade	`api/routes.py`	Single REST entry point
Singleton	`ingestion/pipeline.py`	`get_embedding_model()` loaded once

Table 9: Design Patterns Implemented in the Prototype

Reflections and Lessons Learned

Architectural Decisions

The Microkernel architecture proved its value repeatedly during development. When Ollama is unavailable, the system gracefully degrades to syntactic + semantic retrieval without any special error handling — the fault isolation boundary at the Orchestrator level handles it automatically. Adding conversational query rewriting also fit naturally into the Orchestrator without touching any plugin code, confirming that the Facade + Microkernel boundary was drawn at the right level.

The decision to implement the Semantic Retriever with adaptive behaviour — direct ChromaDB query for small corpora, Chain of Responsibility for large ones — was driven by the observation that the cascading filter's heuristics (title overlap, summary similarity) lack statistical power over tiny corpora. The switch point (100 documents) keeps the small-corpus experience accurate without sacrificing the architectural intent of the CoR pattern at scale.

Technology Choices

- **ChromaDB** was an excellent choice for the vector store: embedded, zero-config, persistent, and supports `$in` metadata filtering needed for the Chain of Responsibility's ContentSim stage.
- **sentence-transformers** with `all-MiniLM-L6-v2` provides strong semantic quality at minimal computational cost with no GPU requirement.
- **uv** significantly improved Python dependency management speed compared to pip.
- **pdfplumber** handles PDF text extraction cleanly, with graceful table fallback.
- **SSE (Server-Sent Events)** over WebSockets was the right choice for streaming: unidirectional, HTTP/1.1 compatible, no upgrade handshake, and directly supported by FastAPI's `StreamingResponse`.

Challenges

- The Chain of Responsibility filter thresholds required careful tuning. An aggressive `TitleMatchHandler` threshold discards relevant documents when the user's phrasing differs from the document title. The current implementation falls through to returning the top candidates by corpus size when no title overlap exists.
- The BM25 index rebuilds from SQLite on every query. This is acceptable for the current corpus size ($\sim$5–50 documents) but would need to be replaced with an event-driven incremental update in production.
- Conversational query rewriting adds one full LLM round-trip before retrieval. For long conversations the rewrite prompt grows; capping history to the last 6 messages keeps this bounded.
- Agentic retrieval quality depends on the SLM's instruction-following ability. Smaller models occasionally return non-numeric output instead of document IDs, requiring robust regex parsing as a fallback.

Future Improvements

- **Advanced OCR pipelines** — the current Tesseract-based OCR handles scanned PDFs and images, but could be extended with layout-aware models (e.g., LayoutLM) for complex

document structures.

- **Event-driven BM25 index updates** — rebuild incrementally on document upload-/delete rather than on every query.
- **Role-based access control** — separate admin (upload/delete) from read-only user access.
- **Graph RAG** as a fourth retrieval strategy, demonstrating extensibility by adding one new file + one factory line with no Orchestrator changes.
- **Persistent conversation storage** — currently conversation history lives only in the browser; persisting it server-side would enable cross-session continuity.

Individual Contributions

Team Member	Roll Number	Contributions
Aviral Gupta	2023111023	Task 1 (Requirements and Subsystems), Task 2 (Stakeholders, ADRs)
Sreemaee Akshathala	2025701028	Task 1 (Requirements and Subsystems), Task 2 (Stakeholders, ADRs)
Arihant Tripathy	2023111026	Task 3 (Tactics, Patterns), Task 4 (Prototype Implementation and Architecture Analysis)
Inesh Dheer	2023111010	Task 3 (Architectural Tactics, Implementation Patterns)
Mohit Kumar Singh	2023111021	Task 4 (Prototype Implementation and Architecture Analysis)

Table 10: Individual Contributions

References

1. IEEE 42010:2011 — Systems and software engineering — Architecture description.
2. M. Nygard, "Architecture Decision Records," 2011.
3. L. Bass, P. Clements, R. Kazman, *Software Architecture in Practice*, 4th ed., 2021.
4. E. Gamma et al., *Design Patterns: Elements of Reusable Object-Oriented Software*, 1994.
5. S. Robertson, N. Walker, "Okapi at TREC-3," NIST Special Publication, 1995 (BM25).
6. N. Reimers, I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," EMNLP 2019.
7. ChromaDB Documentation — https://docs.trychroma.com/
8. FastAPI Documentation — https://fastapi.tiangolo.com/

GitHub Repository: https://github.com/Arihant25/aamsir